

RL Learning · 强化学习

Part 3 — CarRacing PPO in Practice: From Formulas to Code That Runs

Continuous Gaussian Policy · GAE Boundary Handling · Implementation Traps in the PPO Loss · Law of Large Numbers · Full Error Log

Contents

- 0 What This Part Records
- 1 Project Structure and Division of Labour
- 2 The Environment Pipeline: Pixels, Frame Stack, Action Space
- 3 Exercise 1 · The Continuous Gaussian Policy (act / ev)
- 4 Exercise 2 · GAE: The Main Battleground
- 5 Exercise 3 · The PPO Loss: Right Math, Six Implementation Traps
- 6 The Law of Large Numbers: Why Monte Carlo Works (the foundation of everything above)
- 7 Two Bugs in the Scaffolding Itself
- 8 Running It: Compute, Time, Speed-ups
- 9 Diagnostic Experiments
- 10 Open Issues and Next Steps
- Appendix A Complete Working Code

0 What This Part Records

Part 1 covered the modelling and solution path from RLHF to PPO; Part 2 was five diagnosis cases. This part is different: it records the full process of **turning the formulas derived earlier into code that actually runs**. The environment is Gymnasium's `CarRacing-v3` — 96×96 pixel input, three-dimensional continuous action, top-down racing.

It deserves its own part because there is a layer between "the math is right" and "the code is right". That layer appears in no paper; it lives only in implementation details: off-by-one array lengths, the semantics of a termination flag, traversal direction, tensor dtype, and the difference between a scalar and a vector. In this round of practice, **almost nothing went wrong in the mathematics, and everything went wrong in that layer**. So this part deliberately preserves every incorrect version and the correction that followed, rather than presenting only the final answer — the final answer is a dozen lines; the debugging path is what was actually learned.

Ground rules. The whole process followed a fixed division of labour: **I write, the AI only judges correctness** — no hints, no answers, no predicting bugs, no reverse-engineering my approach. This rule broke down once, on the final cell of the GAE recursion (four consecutive rounds of "incorrect" with no forward progress); § 4.9 records that failure honestly, along with the resulting adjustment.

1 Project Structure and Division of Labour

The notebook has 17 cells, split as follows:

Module	Content	Author
Environment pipeline	Pixel preprocessing, frame stacking, action transform, <code>make_env</code>	AI (given)
Network scaffolding	CNN backbone (Nature-DQN config), automatic flatten-dimension inference, three heads	AI (given)
<code>TODO 1</code>	<code>act / ev</code> : sampling and evaluation under a continuous Gaussian policy	Me
<code>TODO 2</code>	<code>compute_gae</code> : the backward advantage recursion	Me
<code>TODO 3</code>	<code>ppo_update</code> : clip surrogate + value loss + entropy	Me
Training loop	Rollout → GAE → update → record GIF	AI (given)

The design intent is explicit: everything **previously derived** is left blank; everything that is **pure engineering scaffolding** is provided. That way, getting stuck always signals a gap in understanding, not noise from a broken install or a miscomputed dimension.

2 The Environment Pipeline: Pixels, Frame Stack, Action Space

2.1 Pixel preprocessing

```
def preprocess(f):
    g = np.dot(f[...,:3],[0.299,0.587,0.114])[6:90,6:90]
    return (g/255.).astype(np.float32)
```

Three things happen: RGB to greyscale (ITU-R BT.601 luminance weights — green dominates because the human eye is most sensitive to it), cropping to 84×84 (the bottom of the original 96×96 frame is the dashboard, low information density), and normalisation to $[0,1]$. The 84×84 size is not arbitrary: it is the standard Nature-DQN input size, which the CNN configuration below assumes.

2.2 Frame stacking: why four frames

```
class FrameStack:
    def __init__(s,e,k=4): s.e=e; s.k=k; s.f=deque(maxlen=k)
```

The core reason is that **a single frame is not a Markov state**. Given a still photograph of the car, you cannot tell which way it is heading, how fast it is going, or whether it is sliding — those quantities are velocity and acceleration, readable only from differences between adjacent frames. An MDP assumes the current state contains everything needed for the decision; a single frame violates that assumption, and stacking four frames approximately restores it.

Why four and not two or eight: two frames give velocity but not acceleration; eight adds little information while doubling memory and compute. Four is the empirical value from the DQN paper and has since become the default.

2.3 Action space and to_env_action

CarRacing's action is three-dimensional and continuous: steering $[-1, 1]$, throttle $[0, 1]$, brake $[0, 1]$. The network emits unconstrained Gaussian samples, which must be mapped into the legal range:

```
def to_env_action(a):
    return np.array([np.tanh(a[0]), np.clip(a[1],0,1), np.clip(a[2],0,1)], np.float32)
```

A theoretical approximation is buried here; § 10 expands on it. What gets stored in the buffer is the **pre-transform** action, so `act` and `ev` compute log-probabilities on the same quantity — internally consistent, nothing miscomputed. But it also means the policy applies no Jacobian correction for the tanh squash (the standard treatment is a tanh-Gaussian with an added $\log(1 - \tanh^2)$ term). This is a deliberate simplification and does not prevent training.

3 Exercise 1 · The Continuous Gaussian Policy (act / ev)

3.1 Why a Gaussian rather than a softmax

A discrete action space can emit one probability per action and normalise with a softmax. A continuous action space has infinitely many actions and cannot be enumerated, so the only option is to **parameterise a distribution**. The Gaussian is the natural choice: two parameters, a closed-form log-density, cheap sampling, and support for reparameterisation.

The three heads map to: `mean_head` for the mean (state-dependent), `log_std` as a **learnable parameter independent of state**, and `v_head` for the state value. Making $\log \sigma$ state-independent is common practice — it acts as a global exploration scale that contracts over training. Parameterising $\log \sigma$ rather than σ guarantees positivity and keeps the optimisation scale sane.

3.2 act versus ev: a trap encountered earlier

```
def act(self, x):                                # sampling phase (no_grad)
    mean, std, value = self.forward(x)
    dist = torch.distributions.Normal(mean, std)
    a = dist.sample()
    return a, dist.log_prob(a).sum(-1), value

def ev(self, x, a):                              # update phase (needs gradients)
    mean, std, value = self.forward(x)
    dist = torch.distributions.Normal(mean, std)
    return dist.log_prob(a).sum(-1), dist.entropy().sum(-1), value
```

The key distinction: **act produces** an action; **ev evaluates** an action that already exists. An earlier version of **ev** contained `a = dist.sample()`, overwriting the passed-in action — a bug I caught myself. The consequence is insidious: the code runs fine, but the importance ratio ρ becomes the ratio between two independent samples, entirely decoupled from PPO's semantics. Training drifts slowly and nothing ever raises an error.

3.3 Where `sum(-1)` comes from

The three action dimensions use a diagonal Gaussian (independent per dimension), so the joint density is a product and its logarithm a sum:

$$\log \pi_{\theta}(a \mid s) = \sum_{i=1}^3 \log \mathcal{N}(a_i; \mu_i(s), \sigma_i)$$

So `.sum(-1)` is not a casual reduction — it encodes the independence assumption. The same applies to the entropy.

4 Exercise 2 · GAE: The Main Battleground

4.1 Definition

The one-step TD residual:

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

GAE is an exponentially weighted average of multi-step advantage estimators, written as a weighted sum of δ :

$$A_t^{\text{GAE}(\gamma, \lambda)} = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}$$

The equivalent recursive form — the direct basis for the code:

$$A_t = \delta_t + \gamma \lambda A_{t+1}$$

Keep the two decay factors distinct: γ belongs to the MDP and determines how far into the future still counts; λ is a hyperparameter of the estimator and governs the bias-variance trade-off. $\lambda = 0$ reduces to one-step TD (low variance, high bias, full trust in V); $\lambda = 1$ reduces to Monte Carlo (unbiased, high variance, full trust in realised returns).

4.2 What dones actually is

The signature is `compute_gae(rews, vals, dones, gamma, lam)`, with these lengths:

Array	Length	Meaning
<code>rews</code>	T	Immediate reward at step t
<code>dones</code>	T	Whether the episode ended after step t
<code>vals</code>	$T+1$	$V(s_0) \dots V(s_{T-1})$ plus a trailing bootstrap $V(s_T)$

The extra entry comes from one additional forward pass after the rollout ends:

```
with torch.no_grad():
    _, lv = model.act(torch.tensor(o, ...).unsqueeze(0))
    V.append(float(lv))
```

4.3 One rollout contains many episodes

This is the key to reading `dones` correctly. The sampling loop runs for a **fixed number of steps**, not a fixed number of episodes:

```
for _ in range(n_steps):          # n_steps = 2000, not one episode
    ...
    D.append(float(t or tr))
    if t or tr: o, _ = e.reset()  # reset in place, keep filling to 2000
```

So a rollout normally contains **several complete episodes** — an early CarRacing policy often crashes off the track within a few dozen steps, so a dozen or more episodes inside 2000 steps is routine. Every `dones[t]=True` marks an episode boundary, and **the last one is not special**: it may be True (an episode happened to end at the boundary) or False (an episode was cut off mid-flight).

A judgement that was corrected. My initial claim was: "if the last element of `dones` is True the game ended, otherwise it was truncated." That reverses the implication. In fact **truncation is signalled by the last done being False** — the episode was still running when the step budget ran out. Position cannot determine semantics; only semantics can.

4.4 terminated versus truncated: a real source of bias

The modern Gymnasium API splits episode endings into `terminated` (the environment's own terminal condition — a crash, a completed task) and `truncated` (an external time limit). The

corresponding s_{t+1} has entirely different status in each case:

Case	Value of s_{t+1}	Correct handling
terminated	No future exists; $V(s_{t+1}) = 0$	Zero out the bootstrap
truncated	The state still has value; we simply stop looking	Still bootstrap ; zeroing introduces bias
Rollout boundary (done=False)	Same as above	Bootstrap with $V(s_T)$

But the sampling code writes:

```
D.append(float(t or tr))          # ← terminated and truncated merged
```

while the seventh markdown cell instructs that "at a done step, both $\gamma V(s_{t+1})$ and the recursion should be multiplied by $(1 - \text{done})$ ". **The two are inconsistent**: following the instruction treats truncation as genuine termination and wrongly zeroes the bootstrap.

This is not a rare path. The TimeLimit for CarRacing-v3 is 1000 steps while `n_steps=2000`, meaning **every rollout hits truncation at least once or twice**. Each time, a state that genuinely has value is estimated as zero, and the value function is systematically depressed. Nothing errors, nothing produces NaN, nothing crashes — it just learns slightly slower and slightly wrong. This is the canonical shape of a "runs fine but learns wrong" defect, and among the hardest classes of bug to catch in code review.

The treatment in this exercise: complete the blank under the signature's assumption (`done`s means genuine termination) and get it running; the bias is left as a follow-up (see § 10). Fixing it requires a signature change — an extra array, or a precomputed `next_value_mask`.

4.5 Why it must be computed at rollout level, not mini-batch level

The recursion $A_t = \delta_t + \gamma \lambda A_{t+1}$ depends on the **temporally adjacent** element. Mini-batches are drawn at random after shuffling; once shuffled, A_{t+1} is simply not in the batch. So the ordering is rigid:

```
Collect a full rollout (temporal order intact)
  → compute GAE once; freeze adv / ret into tensors in the buffer
  → shuffle, slice mini-batches, run several epochs
  → each batch merely reads values already computed
```

This also explains why `ppo_update` receives `adv` and `ret` as **precomputed constants** rather than recomputing them — they stay fixed across epochs, which is precisely what makes PPO's "multiple updates on one batch of data" possible.

4.6 Why backward

Separate the two quantities:

- **δ has no directional constraint.** δ_t uses only `rews[t]`, `vals[t]`, `vals[t+1]`, `done[t]` — all local. The whole array can be vectorised; forward or backward both work.
- **A must go backward.** A_t depends on A_{t+1} . Scanning forward, by the time you reach $t = 0$, A_1 has not been computed.

Forward is not impossible, merely more expensive: either expand $\sum (\gamma\lambda)^l \delta_{t+l}$ and accumulate per index (rescanning the tail for every t , $O(T^2)$), or run a backward pass first and then use it forward (a detour). One backward sweep is $O(T)$. **The difference is complexity, not correctness.**

4.7 Two cut points, not one

When `done[t]=True`, two distinct things must be cut:

1. **The bootstrap inside δ_t :** drop $\gamma V(s_{t+1})$, since s_{t+1} belongs to a different episode. Then $\delta_t = r_t - V(s_t)$.
2. **The recursive accumulation:** drop $\gamma\lambda A_{t+1}$, otherwise **the next episode's advantage leaks backward into the tail of the previous one**. Then $A_t = \delta_t$.

The scaffolding suggests multiplying by $(1 - \text{done}_t)$; I used branches instead. The two are **the same thing written differently**, not two approaches — at one point I claimed the handling was "unrelated to $1 - \text{done}$ ", which overstated the case: not needing the multiplicative form is fine, calling it unrelated is not.

4.8 The final cell: $A_{T-1} = \delta_{T-1}$

This was the longest sticking point in the exercise and deserves full expansion. Expanding the definition at $t = T - 1$:

$$A_{T-1} = \delta_{T-1} + (\gamma\lambda) \delta_T + (\gamma\lambda)^2 \delta_{T+1} + \dots$$

The `delta` array is filled by `for t in range(T)`, so its largest index is $T - 1$. δ_T and everything after it **do not exist**, and all terms with $l \geq 1$ vanish:

$A_{T-1} = \delta_{T-1}$

If one insists on assigning A_T a value, it is **0** — an empty sum. That is exactly what `last = 0.0` in the scaffolding means: it is the rolling variable " A_{t+1} ", and initialising it to zero declares $A_T = 0$. With it, the final step follows the general formula, $\delta_t + \gamma\lambda \cdot 0 = \delta_t$, and **the special case disappears on its own**.

The trailing bootstrap has already been consumed; do not count it twice. The boundary value $V(s_T)$ enters through $\gamma \text{vals}[T]$ inside $\text{delta}[T-1]$. Adding $\text{vals}[-1]$ again at $\text{adv}[T-1]$ double-counts it — and the units are wrong anyway: in the structure $\gamma\lambda \cdot X$, the slot for X admits an advantage, never a value.

4.9 A record of four failed attempts

This one cell was written incorrectly four times in a row, each attempt pointing in the same direction: "find some number inside `vals` to plug in." Recorded faithfully:

Version	Code	What was wrong
v1	<code>adv[t] = vals[:-1]</code>	Shape error: an array of length T assigned to a scalar slot.
v2	<code>adv[t] = vals[-1]</code>	Shape fixed, value wrong: $V(s_T)$ is a value, not an advantage.
v3	<code>adv[t] = vals[T-1]</code>	Same again — still indexing into <code>vals</code> , which contains no advantages at all.
v4	<code>adv[t] = delta[t] + $\gamma\lambda \cdot \text{vals}[-1]$</code>	Structure half right, but $\gamma\lambda$ still multiplies a value; and $V(s_T)$ was already used in <code>delta[T-1]</code> , so it is double-counted.
Correct	<code>adv[t] = delta[t]</code>	—

One further correction landed between v1 and v2: initially only the "rollout tail" special case was handled, missing the fact that **every** `done[t]=True` requires a cut. Had an episode ended at $t = 500$, the original code would still have multiplied in A_{501} — an advantage from the next episode.

An adjustment to the collaboration rules. Across those four rounds the AI replied only "incorrect, return to the definition." The result was that I guessed and it judged, with no information gain. This is the failure mode of "judge correctness, give no hints" when **the diagnostic signal is too thin**: the judgements were right (all four versions would break training), but repeating the same feedback four times does not constitute a functioning feedback loop. Giving the derivation and the answer directly on the fifth round resolved it immediately.

Conclusion: "no answers" suits situations where my direction is right and only the last step is missing. When two consecutive attempts deviate systematically, the correct move is to switch to giving the derivation. This applies to all future derivation work.

4.10 Final implementation

```
def compute_gae(rews, vals, dones, gamma=0.99, lam=0.95):
    T = len(rews); adv = np.zeros(T, np.float32)
    delta = []
    for t in range(T):
        #  $\delta$ : forward, vectorisable
        if dones[t] == False:
            delta_t = rews[t] + gamma * vals[t+1] - vals[t]
        else:
            delta_t = rews[t] - vals[t]
            # cut ①: bootstrap
        delta.append(delta_t)

    for t in range(T-1, -1, -1):
        # A: must be backward
        if t+1 == T:
            adv[t] = delta[t]
            #  $A_T = 0$  (empty sum)
        elif dones[t] == True:
            adv[t] = delta[t]
            # cut ②: recursion
        else:
            adv[t] = delta[t] + gamma * lam * adv[t+1]

    return adv, adv + vals[:-1]
```

Optional tightening: a `last` rolling variable collapses three branches into two and merges the two loops into one (computing δ_t in place during the backward sweep). Functionally equivalent; the expanded form is kept here to mirror the definition.

4.11 The correct name for the second return value

The second return value `adv + vals[:-1]` is the regression target for the value head. Rearranging $A = Q - V$ gives $Q = A + V$.

A statement that was corrected. The AI initially described this as "rearranging $A = R - V$, hence $R = A + V$ " — wrong: the rearrangement yields $Q(s_t, a_t)$, not the discounted return R_t . More precisely, this quantity is the **λ -return**. Adding V back to GAE's λ -weighted sum reproduces exactly the TD(λ) return. At $\lambda = 1$ it degenerates to the Monte Carlo return; at $\lambda = 0$ to the one-step TD target $r_t + \gamma V(s_{t+1})$. Calling it "return" is standard, but equating it with the ordinary discounted return is not.

It serves as a regression target for V because $V(s) = \mathbb{E}_a[Q(s, a)]$, and only one action was actually sampled — so this is a **single-sample estimate** of $V(s_t)$.

5 Exercise 3 · The PPO Loss: Right Math, Six Implementation Traps

5.1 The three terms and their signs

$$L = -\mathbb{E}[\min(\rho_t A_t, \text{clip}(\rho_t, 1 - \epsilon, 1 + \epsilon) A_t)] + c_1 \text{VL} - c_2 \mathcal{H}$$

with $\rho_t = \exp(\log \pi_{\text{new}} - \log \pi_{\text{old}})$ the importance sampling ratio. Each sign has its own origin:

- **Policy term negative:** the objective is to **maximise** the surrogate, while the optimiser minimises the loss.
- **Value term positive:** MSE is already something to minimise.
- **Entropy term negative:** we want entropy **high** (to preserve exploration), so it is subtracted. $c_2 = 0.01$ is the standard value; setting it to zero is the exploration experiment in § 9.

The clip applies to ρ , not A : it constrains how far the new policy may drift from the old, independently of advantage magnitude. This was derived in Part 1.

5.2 Six problems in the first version

The mathematical structure and all three signs were correct on the first attempt; every problem was at the implementation layer. Ordered by how loudly they fail:

#	Problem	Consequence
1	<code>A</code> undefined (should be <code>adv[b]</code>)	<code>NameError</code> , immediate crash
2	<code>old_logp</code> not sliced	<code>(N,)</code> against <code>(mb,)</code> broadcasts to <code>(mb,N)</code> — silent, entirely wrong
3	Bare <code>exp</code> / <code>min</code> / <code>np.mean</code>	Different semantics from the torch versions; <code>np.mean</code> on a tensor severs the gradient
4	<code>sum((v-ret)*(v-ret), axis=-1)</code>	A sum of squares , not a mean — larger by a factor of <code>mb</code> ($128\times$), so c_1 stops meaning anything
5	<code>ent</code> not averaged	The loss stays a vector; <code>.backward()</code> raises
6	Policy term written with <code>+</code>	Gradient direction fully reversed; the agent actively seeks low-advantage actions and training collapses

Point 3 is worth isolating: `torch.min` is an **element-wise minimum**, while Python's built-in `min` **compares two objects and returns one of them**. Applied to tensors, the latter yields "all of tensor A" or "all of tensor B" — completely different semantics, and not necessarily an error.

Point 6 was **introduced** during the second revision: the first version's `loss - mean(...)` was already correct, but my feedback "do not add another negative sign" was ambiguous (the intent was "do not flip a sign that is already negative"), which prompted the change to `+`. A regression caused by imprecise feedback wording.

5.3 Final implementation

```
def ppo_update(model, opt, obs, acts, old_logp, adv, ret,
               clip=0.2, c1=0.5, c2=0.01, epochs=4, mb=128):
    adv = (adv - adv.mean())/(adv.std()+1e-8)      # normalise once per rollout
    N = len(obs)
    for _ in range(epochs):
        idx = torch.randperm(N, device=device)
        for s in range(0, N, mb):
            b = idx[s:s+mb]
            new_logp, ent, v = model.ev(obs[b], acts[b])

            loss = c1 * F.mse_loss(v, ret[b])
            loss = loss - c2 * torch.mean(ent)
            ratio = torch.exp(new_logp - old_logp[b])
            loss = loss - torch.mean(torch.min(
                ratio * adv[b],
                torch.clip(ratio, 1-clip, 1+clip) * adv[b]))

    opt.zero_grad(); loss.backward(); opt.step()
```

5.4 Advantage normalisation: why it appears in no derivation

A well-aimed question — **it genuinely appears in no derivation**. It is absent from the body of the PPO paper; it entered through the OpenAI baselines implementation and was widely copied afterwards. The two halves deserve separate verdicts:

- **Subtracting the mean** has theoretical backing. The policy gradient is invariant to any action-independent baseline, $\mathbb{E}[\nabla_{\theta} \log \pi \cdot b] = 0$, so subtracting the mean **leaves the expected gradient unchanged** and only reduces variance. This is the baseline argument.
- **Dividing by the standard deviation** has **no** such invariance; it does change the gradient, equivalent to scaling the learning rate by an adaptive $1/\sigma$. The purpose is to keep gradient magnitudes from drifting with reward units and training stage. CarRacing's reward scale differs from CartPole's by an order of magnitude; without the division, one set of hyperparameters (`lr=3e-4` , `clip=0.2`) could not serve both.

So it is biased, but the bias serves the goal of making the clip threshold and learning rate transferable across environments, and in practice the benefit outweighs the cost. ICLR 2022's *What Matters in On-Policy RL* evaluated this family of tricks systematically; advantage normalisation is among the few with consistently positive effect.

An implementation fork. This code normalises **outside** the epoch loop — once per rollout. Some implementations (CleanRL by default) normalise per mini-batch, which gives each batch its own mean and variance, injecting extra noise and estimating poorly at small batch sizes. Both are in use; the difference is worth knowing.

Note that `ret` is **not** normalised: it is the regression target for the value head, and normalising it would put it on a different scale from `v`.

6 The Law of Large Numbers: Why Monte Carlo Works

Everything above — GAE, the PPO loss, advantage normalisation — rests on one operation: **replacing an expectation that cannot be computed with the arithmetic mean of finitely many samples**. What licenses that step is the law of large numbers. It is not a trick used in one place but the foundation of the entire route; remove it and every `.mean()` written above loses its meaning.

6.1 The problem: the expectation is not computable

The policy gradient objective is

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) A_t \right]$$

This expectation ranges over **all possible trajectories**. CarRacing's actions are three-dimensional and continuous, an episode runs to a thousand steps, and the transition probability $p(s' | s, a)$ is given implicitly by a physics engine with no analytic form available. The expectation is therefore **neither enumerable nor integrable**.

The only viable route: while we cannot compute $\mathbb{E}$, we **can sample from the distribution** (letting the agent run *is* sampling), so we approximate with a sample mean. That is Monte Carlo, and the proof that a sample mean may stand in for an expectation is the law of large numbers.

6.2 The weak law and its proof

Let $X_1, \dots, X_n$ be i.i.d. with $\mathbb{E}[X_i] = \mu$, $\text{Var}(X_i) = \sigma^2 < \infty$, and sample mean $\bar{X}_n = \frac{1}{n} \sum X_i$. Then for any $\varepsilon > 0$:

$$\lim_{n \rightarrow \infty} \mathbb{P}(|\bar{X}_n - \mu| \geq \varepsilon) = 0$$

The proof takes two steps via Chebyshev's inequality (derived separately in the mathematics series).

First, the variance of the sample mean. By independence, variances add:

$$\text{Var}(\bar{X}_n) = \frac{1}{n^2} \sum_{i=1}^n \text{Var}(X_i) = \frac{\sigma^2}{n}$$

Second, substitute into Chebyshev, $\mathbb{P}(|X - \mu| \geq \varepsilon) \leq \text{Var}(X)/\varepsilon^2$:

$$\mathbb{P}(|\bar{X}_n - \mu| \geq \varepsilon) \leq \frac{\sigma^2}{n \varepsilon^2} \xrightarrow{n \rightarrow \infty} 0$$

Note the shape of this bound; it dictates the direction of every engineering measure that follows. The bound on the deviation probability is $\sigma^2/(n\varepsilon^2)$ — **variance** in the numerator, **sample size** in the denominator. Only two routes make the estimate more reliable: **increase n** (more samples, expensive) or **decrease σ^2** (variance reduction, free).

The entire toolkit inside PPO — baselines, GAE's λ , advantage normalisation, parallel environments — **falls entirely along these two routes**. That is not coincidence; it is an engineering agenda read directly off this inequality.

6.3 How many times this code invokes the law

Laid out explicitly, every `.mean()` written above is one instance of substituting a sample mean for an expectation:

Code location	Expectation being replaced	Sample size n
<code>torch.mean(torch.min(...))</code>	$\mathbb{E}_{s,a \sim \pi_{\text{old}}}[\min(\rho A, \text{clip} \cdot A)]$	<code>mb = 128</code>
<code>F.mse_loss(v, ret[b])</code>	$\mathbb{E}[(V_\theta(s) - \text{target})^2]$	<code>mb = 128</code>
<code>torch.mean(ent)</code>	$\mathbb{E}_s[\mathcal{H}(\pi(\cdot s))]$	<code>mb = 128</code>
<code>adv.mean()</code> / <code>adv.std()</code>	First and second moments of the advantage	<code>N = 2000</code>
<code>adv + vals[:-1]</code> (λ -return)	$V(s_t) = \mathbb{E}_a[Q(s_t, a)]$	<code>n = 1</code>

The last row deserves a pause: the value head's regression target is a **single-sample** estimate — one action was sampled, and its λ -return is taken as the estimate of $V(s_t)$. At $n = 1$ the law of large numbers offers no guarantee at all; the variance of a single sample is σ^2 itself.

It nonetheless works because averaging happens at another level: the same state (or similar states) is revisited repeatedly during training, and the network's own fitting smooths across samples. In other words, **the expectation is not taken inside any single target; it is taken across the training process by gradient descent**. This also explains why value loss is inherently noisier than policy loss, and why $c_1 = 0.5$ damps its weight.

6.4 The rate of convergence comes from the CLT

The law of large numbers says convergence happens as $n \rightarrow \infty$; it says nothing about speed. The central limit theorem supplies that:

$$\sqrt{n}(\bar{X}_n - \mu) \xrightarrow{d} \mathcal{N}(0, \sigma^2) \quad \implies \quad \bar{X}_n - \mu \sim \frac{\sigma}{\sqrt{n}}$$

The error shrinks as $1/\sqrt{n}$ — a slow rate: **a tenfold gain in precision costs a hundredfold increase in samples**. This is the mathematical root of RL's poor sample efficiency, and the reason that, at a fixed sampling budget, reducing σ is almost the only economical lever.

Placing the two operations of § 5.4 back into this frame clarifies their status:

- **Subtracting the mean (baseline):** $\mathbb{E}[\nabla_{\theta} \log \pi \cdot b] = 0$, so μ is unchanged and σ shrinks — strictly along the "reduce σ^2 " route, at no cost in bias. The one free lunch.
- **Dividing by the standard deviation:** changes the scale of μ , so it is not a legitimate operation inside this frame — it is a purely engineering adaptive learning rate.
- **GAE's λ :** a continuous slider between variance and bias by construction — $\lambda = 1$ unbiased and high-variance, $\lambda = 0$ biased and low-variance.

6.5 The CLT's second role: why a Gaussian is a reasonable assumption

Beyond the engineering reasons (few parameters, closed-form density), the Gaussian policy has statistical support: the return is a **sum of many rewards**, $R_t = \sum_l \gamma^l r_{t+l}$. Sums of random quantities tend toward normality under weak dependence, by the CLT. Modelling return-related quantities as Gaussian is therefore not merely convenient but asymptotically justified.

The same line explains why the value loss uses MSE. Assume $\text{target} \sim \mathcal{N}(V_{\theta}(s), \sigma^2)$ and write the negative log-likelihood:

$$-\log p = \frac{(\text{target} - V_{\theta}(s))^2}{2\sigma^2} + \log \sigma + \frac{1}{2} \log 2\pi$$

The last two terms are constants under differentiation with respect to θ , and what remains is **MSE**. So MSE is not an arbitrary distance function; it is maximum likelihood under a Gaussian noise assumption. By the same construction, cross-entropy for a discrete policy is maximum likelihood under a categorical assumption — the two come from one mould.

6.6 What happens when the law's premises break

The theorem has premises: **independent and identically distributed** samples, drawn from **the target distribution itself**. Both are fragile in RL, and several of PPO's design choices are precisely repairs:

Broken premise	How it shows up in RL	The repair
Samples from the target distribution	After a few updates π_{θ} is no longer the π_{old} that generated the data; samples are stale	The importance ratio $\rho = \pi_{\theta}/\pi_{\text{old}}$ converts the distribution back
Stability of the converted estimate	When ρ strays far from 1 the variance explodes (the classic pathology of ratio estimators)	Clip ρ into $[1 - \epsilon, 1 + \epsilon]$; and run only 4 epochs before resampling
Independence	Adjacent samples within a trajectory are highly correlated; the effective sample size is far below N	Parallel environments; shuffle before mini-batching

Put differently: **every component PPO adds over naive policy gradient exists to preserve the validity of the Monte Carlo estimate** — keeping it unbiased (importance sampling), controlling

its variance (clip, baseline, GAE), or restoring independence (parallelism and shuffling). Seen this way, PPO stops being a pile of formulas to memorise and becomes a chain of consequences that follows necessarily from "the expectation cannot be computed."

7 Two Bugs in the Scaffolding Itself

7.1 terminated / truncated merged

See § 4.4. The sampling line `D.append(float(t or tr))` is semantically inconsistent with what the instruction cell requires — a defect in the scaffolding, not in the exercises. Completing the blanks under the signature's assumption is correct; the fix is a follow-up.

7.2 dtype: Double versus Float

With all three blanks correct, the first run still failed:

```
RuntimeError: Found dtype Double but expected Float
```

Tracing back: `adv` is created by `np.zeros(T, np.float32)`, hence float32; but `ret = adv + vals[:-1]`, and `vals = np.array(V)` is built from a list of Python floats and defaults to **float64**. float32 + float64 broadcasts to float64, so `rt` is Double while `v` is Float, and `mse_loss` fails during the backward pass.

In the training loop, `ot` / `At` / `Lpt` all specify `dtype=torch.float32` explicitly; only this line omits it:

```
# before
at=torch.tensor(adv,device=device); rt=torch.tensor(ret,device=device)
# after
at=torch.tensor(adv,dtype=torch.float32,device=device)
rt=torch.tensor(ret,dtype=torch.float32,device=device)
```

The signature of this bug class: the error message sits far from the cause (raised inside the C++ autograd engine, caused by a numpy broadcast thirty lines away). The way to find it is to trace the dtype of every input to the failing operator backwards, not to stare at the reported line.

8 Running It: Compute, Time, Speed-ups

8.1 Resource accounting

Colab Pro with a T4 (15 GB):

- **Memory:** not a constraint. The CNN is the small Nature-DQN configuration; a mini-batch of $128 \times 4 \times 84 \times 84$ plus one rollout of 2000 observations ($2000 \times 4 \times 84 \times 84$ float32 $\approx$ 226 MB) leaves ample headroom.
- **Time:** `iterations=300, n_steps=2000` is 600k environment steps, roughly 3–6 hours on a T4.

- **The bottleneck is not the GPU** but the Box2D physics simulation — with a single serial environment the GPU idles most of the time.

8.2 Speed-ups by value for effort

Measure	Expected gain	Cost
Parallel environments (<code>AsyncVectorEnv</code> , 8–16)	5–8×	<code>compute_gae</code> must be rewritten for (T, num_envs), with per-environment done handling
Frame skip (repeat the action 4 frames, accumulate reward)	2–4×	Under ten lines; a well-established gain on CarRacing
Lower <code>n_steps</code> , raise <code>iterations</code>	No change in total time	More frequent updates; earlier visible progress
AMP mixed precision	Limited	This CNN is too small to justify the added complexity

8.3 Observed training curve

```
[before training, flailing] score: -20.6
iter   0 | reward/ep ≈ -53.1
iter  10 | reward/ep ≈ -72.4
iter  20 | reward/ep ≈ -70.4
iter  30 | reward/ep ≈ -61.9
iter  40 | reward/ep ≈  66.4
iter  50 | reward/ep ≈ 104.3    → agent_iter50.gif  score 130.7
```

With the three blanks filled and the dtype fixed, the pipeline runs end to end — no NaN, no errors — and this is a textbook CarRacing curve. Iterations 0–30 hover in negative territory and even dip: CarRacing applies a small per-step time penalty, early policies crash quickly, episodes are short, and almost no track tiles are collected. Meanwhile the value head is learning state values from scratch, so the advantage signal-to-noise ratio is poor and the policy is close to a random walk.

Iterations 30→40 mark the inflection — a jump of 128 points. This step change is characteristic: once the agent acquires the basic behavior of accelerating along the track, the reward structure flips from penalty-dominated to tile-dominated, and the score moves discontinuously rather than gradually. The recorded episode (130.7) sits slightly above the training average (104.3), as expected — recording is a single noise-free evaluation while training samples still carry exploration noise. For reference, CarRacing-v3 is usually considered solved around 900; reaching triple digits by iteration 50 only confirms that the learning signal is intact. Progress slows markedly afterwards and drawdowns of tens of points are normal. What genuinely warrants concern is a long plateau (exploration collapse) or a drop back to negative territory with no recovery (policy collapse, usually accompanied by an exploding ratio).

Two caveats about the log itself. First, `if it%10==0` means only iterations 0/10/20... print; iterations 1 and 2 ran but logged nothing. Second, `reward/ep = sum(R)/max(1,sum(D))` has an inflated denominator because `D` includes truncations, so the value is systematically depressed — usable as a trend, not as an absolute figure.

Note also that the printed quantity is the **episode return**, not the loss — the loss is never printed. And in PPO the loss is **not** expected to decrease monotonically, nor is it a useful diagnostic: the policy term resets to a ratio of 1 whenever `old_logp` is refreshed, the value term *rises* whenever the agent reaches states it has not seen before, and the entropy term (negated) rises as entropy naturally falls. The intuition "falling loss means learning," imported from supervised learning, fails here because the data distribution moves with the model — this is not one fixed optimization problem but a sequence of moving targets. The metrics worth watching are episode return, explained variance, approximate KL, and clip fraction.

9 Diagnostic Experiments

The real learning starts once the code runs. Each item maps to a diagnostic axis from Part 2:

Experiment	Change	What to expect
Exploration collapse	<code>c2</code> $\rightarrow$ 0	Entropy is no longer encouraged; the policy converges prematurely to a suboptimal mode (spinning in place) and stalls
GAE bias/variance	<code>lam</code> $\rightarrow$ 0 / 1	At 0, full trust in V , so an inaccurate early V misleads; at 1, unbiased but high-variance and visibly jittery
Reward misspecification	Wrap the env to reward speed only, ignoring the track	The agent learns to race off-track — a minimal reproducible instance of reward hacking
Generalisation	CarRacing resamples the track on every reset	A generalisation test for free, no extra construction needed
Truncation bias	A/B around fixing <code>t</code> or <code>tr</code>	Differences in the value-loss curve and final score — a direct test of the § 4.4 claim

The third is especially worth doing: it turns the abstract notion of reward misspecification from Part 1 into a phenomenon reproducible in ten lines — usable material for an interview.

10 Open Issues and Next Steps

1. **Separate terminated from truncated.** Either store only terminated at the sampling side (simpler) or extend the signature with a second array (more correct). Run an A/B comparison afterwards — that alone is a complete small experiment.
2. **Jacobian correction for the tanh transform.** The current policy defines its distribution in unconstrained space and squashes externally, without the $\log(1 - \tanh^2 a)$ log-determinant

term. The standard treatment is the tanh-Gaussian (as in SAC). It does not prevent training, but the action distribution behaves inconsistently with theory.

3. **A vectorised GAE for parallel environments.** This is the step from "can write GAE for one trajectory" to "can write production GAE": the arrays become two-dimensional and cut logic becomes per-environment.
4. **Observation normalisation / reward scaling.** Observations are currently only divided by 255 and rewards used raw. Running mean/std normalisation is another family of common tricks, suitable as a controlled experiment.

Appendix A Complete Working Code

A.1 Network (TODO 1 filled)

```
class ActorCritic(nn.Module):
    def __init__(self, act_dim=3):
        super().__init__()
        self.conv = nn.Sequential(
            nn.Conv2d(4,32,8,4), nn.ReLU(),
            nn.Conv2d(32,64,4,2), nn.ReLU(),
            nn.Conv2d(64,64,3,1), nn.ReLU(), nn.Flatten())
        with torch.no_grad():
            n = self.conv(torch.zeros(1,4,84,84)).shape[1]
        self.fc = nn.Linear(n, 512)
        self.mean_head = nn.Linear(512, act_dim)
        self.log_std = nn.Parameter(torch.zeros(act_dim))
        self.v_head = nn.Linear(512, 1)

    def forward(self, x):
        h = F.relu(self.fc(self.conv(x)))
        mean = self.mean_head(h)
        std = self.log_std.exp().expand_as(mean)
        value = self.v_head(h).squeeze(-1)
        return mean, std, value

    def act(self, x):
        mean, std, value = self.forward(x)
        dist = torch.distributions.Normal(mean, std)
        a = dist.sample()
        return a, dist.log_prob(a).sum(-1), value

    def ev(self, x, a):
        mean, std, value = self.forward(x)
        dist = torch.distributions.Normal(mean, std)
        return dist.log_prob(a).sum(-1), dist.entropy().sum(-1), value
```

A.2 GAE (TODO 2)

```
def compute_gae(rews, vals, dones, gamma=0.99, lam=0.95):
    T = len(rews); adv = np.zeros(T, np.float32)
    delta = []
    for t in range(T):
        if dones[t] == False:
            delta_t = rews[t] + gamma * vals[t+1] - vals[t]
        else:
            delta_t = rews[t] - vals[t]
        delta.append(delta_t)
    for t in range(T-1, -1, -1):
        if t+1 == T:
            adv[t] = delta[t]
        elif dones[t] == True:
            adv[t] = delta[t]
        else:
            adv[t] = delta[t] + gamma * lam * adv[t+1]
    return adv, adv + vals[:-1]
```

A.3 PPO update (TODO 3)

```
def ppo_update(model, opt, obs, acts, old_logp, adv, ret,
               clip=0.2, c1=0.5, c2=0.01, epochs=4, mb=128):
    adv = (adv - adv.mean())/(adv.std()+1e-8)
    N = len(obs)
    for _ in range(epochs):
        idx = torch.randperm(N, device=device)
        for s in range(0, N, mb):
            b = idx[s:s+mb]
            new_logp, ent, v = model.ev(obs[b], acts[b])
            loss = c1 * F.mse_loss(v, ret[b])
            loss = loss - c2 * torch.mean(ent)
            ratio = torch.exp(new_logp - old_logp[b])
            loss = loss - torch.mean(torch.min(
                ratio * adv[b],
                torch.clip(ratio, 1-clip, 1+clip) * adv[b]))
            opt.zero_grad(); loss.backward(); opt.step()
```

A.4 The dtype fix in the training loop

```
at = torch.tensor(adv, dtype=torch.float32, device=device)
rt = torch.tensor(ret, dtype=torch.float32, device=device)
```

This part is an edited transcript of a working session. The main derivations and the correction log were cross-checked in both directions, but before citing it formally, re-derive the GAE boundary conditions and the equivalence of the λ -return independently. Items marked "corrected" record cases where each side made an error and was caught by the other.